FBS: FAST BLOB STORAGE

A MINI PROJECT SYNOPSIS SUBMITTED
IN PARTIAL FULFILLMENT OF THE REQUIREMENTS
FOR THE AWARD OF DEGREE OF

BACHELOR OF TECHNOLOGY

In

COMPUTER SCIENCE AND ENGINEERING

SUBMITTED BY

Radhey Kalra (Roll Number: 2023A1R044)

Raghav Nargotra (Roll Number: 033)

Sheershak Sharma (Roll Number: 004)

SUBMITTED TO

Department of Computer Science and Engineering

Model Institute of Engineering and Technology (Autonomous)

Jammu, India

2026

CONTENTS OF SYNOPSIS

1 Project Overview 1
2 Problem Statement 1
3 Objectives of the Project 2
4 Proposed Solution / Methodology 3
5 Key Technologies and Tools 4
6 V.E.T.S Justification 5
7 Expected Outcomes 6
8 Implementation Timeline 6

1. Project Overview

The S3 API, originally introduced by Amazon Web Services, has become the de facto interface for object storage in modern computing. From mobile app backends to machine learning pipelines, applications universally assume an S3-compatible endpoint for storing and retrieving unstructured data. Simultaneously, the self-hosting movement has accelerated: driven by privacy concerns, unpredictable cloud egress costs, and the desire for infrastructure sovereignty, a growing number of developers and hobbyists now run production workloads on single-node hardware — Raspberry Pis, NUCs, and low-end VPS instances. These users need S3-compatible storage, but every existing solution forces a choice between enterprise systems designed for multi-node clusters and lightweight mocks built only for testing. This project presents FBS (Fast Blob Storage), a self-hosted, S3-compatible storage ecosystem that fills this gap: a single-binary Go backend with SQLite metadata, optimized specifically for single-node deployments, paired with a decoupled SvelteKit web dashboard and an OpenTUI terminal dashboard. It targets homelab enthusiasts, developers, and small teams who need production-grade S3 storage without commercial cloud dependencies or the operational overhead of distributed systems.

2. Problem Statement

Existing self-hosted S3-compatible storage solutions fall into two categories, neither of which adequately serves single-node, resource-constrained deployments.

Overengineered distributed systems. Production-grade solutions such as MinIO, Garage, and SeaweedFS are architected for multi-node clusters and carry that overhead even on a single machine. MinIO requires a minimum of four drives for erasure coding, writes a separate metadata file for every object (doubling disk I/O), and consumes 300–500 MB of RAM at idle. SeaweedFS requires coordinating six separate processes — master server, volume server, filer, S3 gateway, WebDAV server, and admin UI — even for a single-node deployment. Garage requires configuring an RPC secret, assigning a cluster layout with zones and capacity, and runs a gossip protocol between nodes even when there is only one. In all three cases, the user pays a significant complexity and resource cost for distributed-system machinery they do not need.

Underengineered testing mocks. Lightweight alternatives such as GoFakeS3, Adobe S3Mock, and s3proxy are explicitly designed for local development and integration testing. GoFakeS3 stores objects in memory by default and states it "should not be used as a production service." Adobe S3Mock deletes all stored data on shutdown unless manually overridden. These tools lack authentication, upload checksum validation, crash recovery, and multipart upload lifecycle management. They do not sanitize object keys

against path traversal attacks and store credentials (if any) in plaintext configuration.

The gap. No existing solution is simultaneously production-usable (persistent storage, authentication, data integrity guarantees), single-binary with zero external dependencies (no separate database server, no multi-process coordination), optimized for single-node performance (zero-copy egress, minimal memory footprint), and equipped with decoupled administrative interfaces (independent web and terminal dashboards that connect remotely). The user is forced to choose between systems that are too heavy for their hardware and tools that are too fragile for their data.

3. Objectives of the Project

- **Design** a decoupled system architecture separating the storage engine, management API, and dashboard components to enable independent development, testing, and deployment.
- **Develop** a high-performance Go backend implementing the complete S3 API with object operations (PUT, GET, DELETE, HEAD), bucket listing (ListObjectsV2), and multipart uploads, with dual-mode authentication (AWS SigV4 and Bearer Token).
- **Implement** a SQLite-based metadata layer with WAL mode, atomic write guarantees, and an in-memory LRU cache for hot metadata queries to minimize disk I/O.
- **Ensure** data integrity through a defined write ordering (data-before-metadata), startup reconciliation of orphaned files, upload checksum validation, and automated cleanup of stale multipart uploads.
- **Secure** the system with object key path sanitization to prevent directory traversal attacks, SHA-256 hashed token storage, and restricted authentication bypass in development mode.
- **Optimize** data egress paths using sendfile() for zero-copy transfers with a dedicated minimal middleware chain, CDN-friendly caching headers, and a Go memory LRU cache to maximize throughput on commodity hardware.
- **Demonstrate** the complete ecosystem by building a SvelteKit web dashboard and an OpenTUI terminal dashboard connected to the backend via the Management API.

4. Proposed Solution / Methodology

The system follows a decoupled microservices-inspired architecture with three independent components communicating over HTTP. Figure 1 shows the complete architecture.

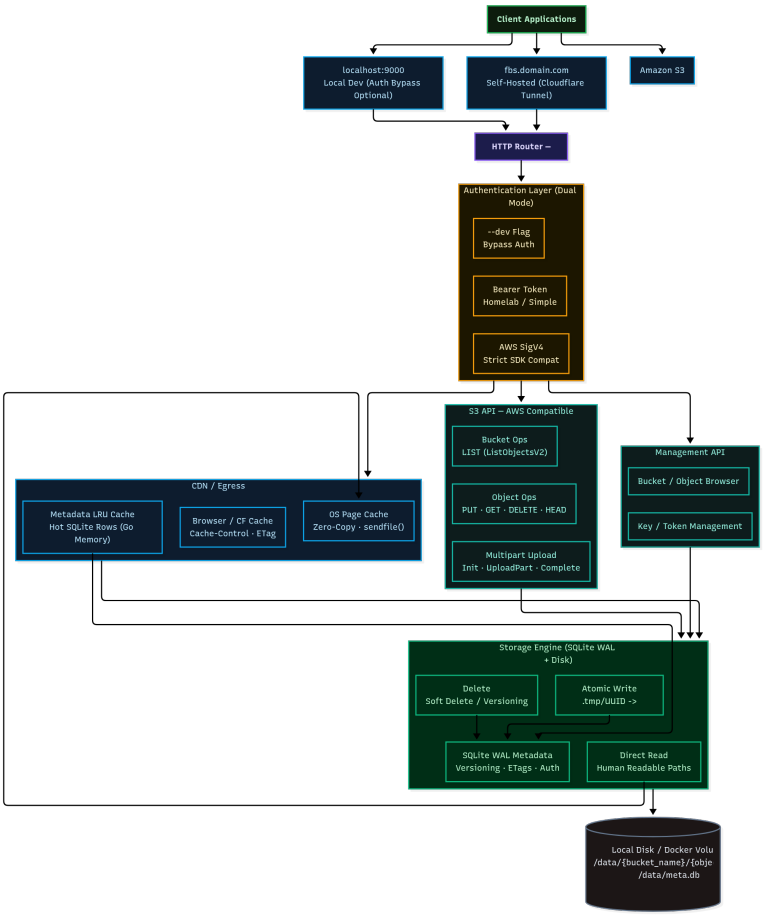

Figure 1: System Architecture of the FBS: Fast Blob Storage

The **Go backend** forms the core, handling all S3 operations and exposing two endpoint groups: the S3 API for object storage and the Management API for administration. The `chi` router handles routing with middleware for logging, panic recovery, and CORS. The S3 `GET` path uses a separate, minimal middleware chain to preserve the `io.ReaderFrom` interface on the response writer, enabling `sendfile()` zero-copy transfers; `http.ServeContent()` handles range requests, ETag, and Last-Modified in a single call.

The storage engine enforces a strict **data-before-metadata write order**: objects are first written to `.tmp/UUID`, atomically renamed to `/data/{bucket}/{key}`, and only then is metadata inserted into SQLite. This makes SQLite the single source of truth — if a metadata row does not exist, the object does not exist. On server startup, a **reconciliation pass** purges incomplete files from `.tmp/` and removes any orphaned data files that lack a corresponding metadata row, ensuring self-healing after crashes.

All object keys undergo **path sanitization**: keys containing traversal sequences (`../`), null bytes, or filesystem-invalid characters are rejected, and intermediate directories are created as needed for keys containing `/`.

Upload integrity is enforced through a **checksum validation pipeline**: `Content-MD5` and `x-amz-checksum-*` headers are verified on upload using a `TeeReader` that hashes data as it streams to disk, rejecting mismatches before metadata is committed. The computed MD5 is stored as the object's ETag. A **background goroutine** periodically purges multipart uploads older than a configurable TTL (default 24 hours), reclaiming disk space from abandoned uploads.

SQLite with WAL mode tracks metadata across four tables: `buckets`, `objects`, `multipart_uploads`, and `keys`, tuned with `synchronous=NORMAL` and `busy_timeout=5000` for optimal single-node write throughput.

The **web dashboard** (SvelteKit + TypeScript + TailwindCSS) connects via Management API, providing a metrics overview, file browser, and access key manager. The **terminal dashboard** (OpenTUI) offers a split-pane file explorer, object metadata views, and real-time metric charts with vim-style keyboard navigation.

Dual-mode authentication serves both SDK clients (AWS SigV4 signature verification) and simpler use cases (Bearer Token for dashboards and scripts). Bearer tokens are stored as **SHA-256 hashes** in SQLite — raw tokens are returned to the user once at creation time and never persisted. A `--dev` flag bypasses authentication but is restricted to `localhost` binding only and logs a startup warning.

5. Key Technologies and Tools

Tool	Description
Go (Golang)	Backend server implementation; goroutines for concurrency, native HTTP performance
chi Router	Lightweight HTTP routing with middleware support
SQLite (WAL Mode)	Embedded metadata storage; WAL enables concurrent reads during writes
HashiCorp LRU	In-memory cache for hot metadata queries and frequently accessed objects
SvelteKit + TypeScript + TailwindCSS	Web dashboard; file-based routing, SSR, responsive UI
OpenTUI	Terminal dashboard; declarative TUI components in TypeScript
Docker	Backend containerization with volume mounts for persistent storage

6. V.E.T.S Justification

V - Viability

The project is highly feasible within the available timeframe and resources. The Go backend compiles to a single static binary with no external runtime dependencies, deployable via a single command. SQLite requires no server process or database administration. The estimated memory footprint is under 50 MB for the backend and sub-100 MB per dashboard, making deployment viable on Raspberry Pi, low-end VPS, and container environments.

E - Engineering Depth

The project spans multiple engineering disciplines. The S3 API implementation requires deep understanding of AWS SigV4 signing (canonical request reconstruction, HMAC-SHA256), HTTP semantics, XML response formatting, and S3 error code conventions. The storage engine involves atomic file operations, directory structure management, path sanitization against traversal attacks, and a data-before-metadata write ordering with startup reconciliation to guarantee consistency between the filesystem and SQLite. Upload integrity is enforced through a TeeReader checksum pipeline validating Content-MD5 at wire speed. The SQLite layer demands schema design, WAL concurrency tuning (`synchronous=NORMAL`, `busy_timeout`), and query optimization. Credential security uses SHA-256 token hashing rather than plaintext storage. Egress uses Linux `sendfile()` syscalls for zero-copy transfers, requiring careful middleware chain design to preserve the `io.ReaderFrom` interface on the response writer.

T - Trend Alignment

The project aligns with the growing self-hosted and homelab movement driven by privacy concerns, cost optimization, and desire for infrastructure control. It follows the cloud-native, API-first design philosophy that dominates modern distributed systems development. The TypeScript-unified stack across web and CLI tooling reflects the industry trend toward full-stack JavaScript/TypeScript ecosystems.

S - Social / Industrial Impact

Homelab enthusiasts gain an accessible, lightweight S3 target for home media servers and self-hosted application stacks. Small teams benefit from a cost-effective alternative to commercial cloud storage, eliminating per-GB egress charges while retaining S3 tooling compatibility. Educational institutions can use it as a hands-on platform for teaching distributed systems, cloud storage concepts, and API design.

7. Expected Outcomes

- A fully functional Go-based S3-compatible storage server as a drop-in replacement for AWS S3.
- Complete S3 API compliance for object CRUD, bucket listing, and multipart uploads, validated against the official AWS SDK.
- Dual authentication: AWS SigV4 verification passing official test suites, and Bearer Token auth (SHA-256 hashed storage) for dashboards.
- Data integrity guarantees through data-before-metadata write ordering, startup reconciliation of orphaned files, upload checksum validation, and automated stale multipart cleanup.
- Security hardening including object key path sanitization against directory traversal, hashed credential storage, and restricted development mode.
- A decoupled Management API with RESTful JSON endpoints.
- A SvelteKit web dashboard for graphical administration and file management.
- An OpenTUI terminal dashboard for keyboard-driven CLI management.
- Egress optimizations including zero-copy transfers via a dedicated minimal middleware chain, LRU metadata cache, and HTTP caching headers.

8. Implementation Timeline

Phase	**Activity**
Phase 1	Problem analysis and requirement study (Core HTTP Server & Routing, SQLite Metadata Layer, Disk Storage Engine)
Phase 2	System design and architecture (Auth Framework, Bearer Tokens, Basic S3 Object Operations)
Phase 3	Implementation and module development (AWS SigV4 Auth, Bucket Operations, Management API)
Phase 4	Testing and evaluation (Multipart Uploads, Egress Caching Optimizations, Building Web & CLI Dashboards)
Phase 5	Documentation and final presentation (End-to-end integration, Bug Fixes, Docker setup, Release)

9. References

[1] Amazon Web Services, "Amazon S3 API Reference [Online]. Available: https://docs.aws.amazon.com/AmazonS3/latest/API/Welcome.html

[2] MinIO, Inc., "MinIO Object Storage [Online]. Available: https://min.io

[3] Amazon Web Services, "Authenticating Requests: Using Query Parameters (AWS Signature Version 4) [Online]. Available: https://docs.aws.amazon.com/AmazonS3/latest/API/sigv4-query-string-auth.html

[4] The Go Authors, "The Go Programming Language [Online]. Available: https://go.dev

[5] Go-Chi, "chi — lightweight, idiomatic router for Go [Online]. Available: https://github.com/go-chi/chi

[6] SQLite Consortium, "Write-Ahead Logging [Online]. Available: https://www.sqlite.org/wal.html

[7] Svelte Team, "SvelteKit [Online]. Available: https://kit.svelte.dev

[8] OpenTUI, "Terminal UI Library for TypeScript [Online]. Available: https://github.com/anomalyco/opentui

[9] Linux Kernel, "sendfile(2) — transfer data between file descriptors. [Online]. Available: https://man7.org/linux/man-pages/man2/sendfile.2.html

[10] HashiCorp, "golang-lru — Golang LRU cache implementation [Online]. Available: https://github.com/hashicorp/golang-lru

[11] Deuxfleurs, "Garage — An open-source distributed object storage service [Online]. Available: https://garagehq.deuxfleurs.fr